NESNE TABANLI PROGRAMLAMA

Laboratuvar Föyü

Hafta 3 — Metotlar ve Nesne İlişkileri

Öğrenci Yönergesi — Adım Adım Uygulama

Bilgi	Değer
Ders	Nesne Tabanlı Programlama (2. Sınıf)
Konu	Metotlar, method overloading ve composition
Ön koşul	Hafta 1–2 (sınıf, nesne, kapsülleme)
Ortam	Visual Studio Community 2026 · .NET 10 (LTS)
Dil	C#
Süre	Yaklaşık 2–3 saat
Teslim	Kod ve konsol ekran çıktısı

1. Bu Haftanın Hedefi

Şimdiye kadar tek tek sınıflar yazdık. Bu hafta iki yeni beceri kazanacağız: metotları daha esnek yazmak (aynı isimle farklı sürümler) ve en önemlisi, bir sınıfın başka sınıfları içermesini sağlamak. Buna composition (birleştirme) denir; gerçek programlar tam olarak böyle, birbiriyle ilişkili nesnelerden kurulur.

2. Kazanımlar

Bu haftanın sonunda şunları yapabilir hâle geleceksiniz:

- Aynı isimli, farklı parametreli metotlar (method overloading) yazmak,
- Bir metottan değer döndürmek ve bu değeri kullanmak,
- Bir sınıfın içinde başka sınıflardan nesneler tutmak (composition),
- List içinde nesne koleksiyonu tutup üzerinde işlem yapmak,
- Birden çok sınıfın birlikte çalıştığı küçük bir sistem tasarlamak.

3. Temel Kavramlar

3.1. Method Overloading (Metot Aşırı Yükleme)

Aynı isimli metodu, farklı parametrelerle birden çok kez tanımlayabilirsiniz. C# hangi sürümü çağırdığınızı parametrelere bakarak anlar. Bu, kullanıma kolaylık sağlar:

```
public void Selamla()
{
    Console.WriteLine("Merhaba!");
}

public void Selamla(string isim)   // aynı isim, farklı parametre
{
    Console.WriteLine($"Merhaba {isim}!");
}
```

3.2. Değer Döndüren Metotlar

void yerine bir tip (int, bool, double...) yazan metotlar return ile değer döndürür. Bu değeri bir değişkene atayabilir veya doğrudan kullanabilirsiniz:

```
public bool StoktaVar(int adet)
{
    return adet > 0;
}
```

3.3. Composition (Nesnelerin Nesneleri İçermesi)

Bir sınıfın alanı, başka bir sınıftan nesne olabilir. Örneğin bir Kutuphane; içinde Kitap nesneleri ve Uye nesneleri tutar. Böylece "bir kütüphane kitaplardan ve üyelerden oluşur" ilişkisini kodda kurmuş oluruz.

"içerir" ilişkisi (has-a)
Composition, bir nesnenin başka nesnelere sahip olması demektir.
Kütüphane bir listeye sahiptir; liste Kitap nesnelerine sahiptir. Buna "has-a" (sahiptir) ilişkisi denir.

4. Uygulama: Kütüphane Otomasyonu

Üç sınıftan oluşan küçük bir sistem kuracağız: Kitap, Uye ve bunları bir arada yöneten Kutuphane. Kütüphane; kitap ekleyecek, ödünç verecek ve iade alacak.

Önce **Hafta3_Kutuphane** adında bir Konsol Uygulaması (C#, .NET 10) oluşturun. Sonra üç ayrı sınıf dosyası ekleyin: **Kitap.cs, Uye.cs, Kutuphane.cs**.

Adım A — Kitap sınıfı

Kitabın bir adı, yazarı ve ödünçte olup olmadığını tutan bir durumu var. Kapsülleme alışkanlığını sürdürüyoruz (geçen haftadan):

```
namespace Hafta3_Kutuphane
{
    class Kitap
    {
        public string Ad { get; set; }
        public string Yazar { get; set; }
        public bool OduncteMi { get; set; }

        public Kitap(string ad, string yazar)
        {
            Ad = ad;
            Yazar = yazar;
            OduncteMi = false;
        }
    }
}
```

Adım B — Uye sınıfı

```
namespace Hafta3_Kutuphane
{
    class Uye
    {
        public string Ad { get; set; }
        public int UyeNo { get; set; }

        public Uye(string ad, int uyeNo)
        {
            Ad = ad;
            UyeNo = uyeNo;
        }
    }
```

```
}
```

Adım C — Kutuphane sınıfı (composition burada)

Kutuphane, içinde bir Kitap listesi tutar (composition). Kitap ekleme metodunu iki farklı biçimde yazarak method overloading de kullanıyoruz: biri hazır bir Kitap nesnesi alır, diğeri ad/yazar alıp kendisi oluşturur.

```
using System;
using System.Collections.Generic;

namespace Hafta3_Kutuphane
{
    class Kutuphane
    {
        private List<Kitap> kitaplar = new List<Kitap>();

        // Overload 1: hazır Kitap nesnesi ekle
        public void KitapEkle(Kitap kitap)
        {
            kitaplar.Add(kitap);
        }

        // Overload 2: ad ve yazar ver, kitabı burada oluştur
        public void KitapEkle(string ad, string yazar)
        {
            kitaplar.Add(new Kitap(ad, yazar));
        }

        public void KitaplariListele()
        {
            Console.WriteLine("--- KITAPLAR ---");
            foreach (Kitap k in kitaplar)
            {
                string durum = k.OduncteMi ? "(oduncte)" : "(rafta)";
                Console.WriteLine($"{k.Ad} - {k.Yazar} {durum}");
            }
        }
    }
}
```

Adım D — Ödünç verme ve iade (üye ile etkileşim)

Kutuphane sınıfının içine, bir üyenin kitap ödünç almasını ve iade etmesini sağlayan metotları ekleyin. Kitabı adına göre listede arıyoruz:

```
public void OduncVer(string kitapAdi, Uye uye)
{
    foreach (Kitap k in kitaplar)
    {
        if (k.Ad == kitapAdi)
        {
            if (k.OduncteMi)
                Console.WriteLine($"'{k.Ad}' zaten odunçte.");
            else
            {
                k.OduncteMi = true;
                Console.WriteLine($"'{k.Ad}', {uye.Ad} adli uyeye verildi.");
            }
            return;
        }
    }
    Console.WriteLine($"'{kitapAdi}' bulunamadi.");
}

public void IadeAl(string kitapAdi)
```

```
{
    foreach (Kitap k in kitaplar)
    {
        if (k.Ad == kitapAdi)
        {
            k.OduncteMi = false;
            Console.WriteLine($"'{k.Ad}' iade alindi.");
            return;
        }
    }
}
```

Adım E — Main içinde sistemi çalıştırın

```
using System;

namespace Hafta3_Kutuphane
{
    class Program
    {
        static void Main(string[] args)
        {
            Kutuphane kutuphane = new Kutuphane();

            // iki farklı overload ile ekleme
            kutuphane.KitapEkle("Suç ve Ceza", "Dostoyevski");
            kutuphane.KitapEkle(new Kitap("Sefiller", "Victor Hugo"));

            Uye uye = new Uye("Ayşe Yılmaz", 101);

            kutuphane.KitaplariListele();
            kutuphane.OduncVer("Sefiller", uye);
            kutuphane.OduncVer("Sefiller", uye);   // zaten oduncte
            kutuphane.IadeAl("Sefiller");

            kutuphane.KitaplariListele();
        }
    }
}
```

Ne görmelisiniz?

--- KITAPLAR ---
Suç ve Ceza - Dostoyevski (rafta)
Sefiller - Victor Hugo (rafta)
'Sefiller', Ayşe Yılmaz adli uyeye verildi.
'Sefiller' zaten oduncte.
'Sefiller' iade alindi.
--- KITAPLAR --- (Sefiller artık tekrar rafta)

5. Kendi Göreviniz (Lab Teslimi)

Uygulamayı aşağıdaki gibi genişletin. Teslim edilecek çalışma budur:

1. Kutuphane sınıfına, ödünçte olmayan (raftaki) kitap sayısını döndüren bir metot ekleyin: public int RaftakiSayisi().
2. En az 4 kitap ekleyin (iki overload'u da kullanın) ve iki farklı üye oluşturun.
3. Birkaç ödünç verme/iade işlemi yapın; olmayan bir kitabı ödünç vermeyi deneyip uyarıyı gösterin.
4. İşlemlerden sonra raftaki kitap sayısını ekrana yazdırın.

6. Sık Yapılan Hatalar

- **List için using unutmak:** List<T> kullanmak için dosyanın başında using System.Collections.Generic; olmalı (modern şablonlarda çoğunlukla otomatik gelir).
- **Listeyi oluşturmadan kullanmak:** private List<Kitap> kitaplar; deyip = new List<Kitap>(); yazmazsanız Add çağrısında NullReferenceException alırsınız.
- **Overload'ları aynı parametreyle yazmak:** İki metot yalnızca isimle değil, parametreleriyle de ayrılmalı; birebir aynı parametre listesi hata verir.
- **Nesneyi eklemeyi unutmak:** new Kitap(...) ile oluşturup listeye Add etmezseniz kitap kütüphaneye girmez.

7. Değerlendirme

Ölçüt	Puan
Proje derleniyor ve çalışıyor	15
Kitap ve Uye sınıfları doğru tanımlı	20
Kutuphane bir Kitap listesi içeriyor (composition)	25
KitapEkle method overloading ile iki sürüm	15
OduncVer / IadeAl doğru çalışıyor	15
RaftakiSayisi metodu doğru değer döndürüyor	10
TOPLAM	100

8. Ödev (Gelecek Haftaya Hazırlık)

Şu an Kitap, Uye ve Kutuphane sınıflarının tümü sıfırdan yazıldı. Ama dikkat ederseniz farklı türden "kişiler" (üye, ileride belki personel) benzer alanlara sahip olurdu: ad, numara...

Ödev: Bir Personel sınıfı daha eklemeyi düşünün ve Uye ile Personel'in ortak alanlarını (Ad gibi) bir kâğıda not edin. **Bu ortak alanları tek bir üst sınıfta toplayabilseydik tekrar yazmaktan kurtulurduk — işte gelecek haftanın konusu bu: kalıtım (inheritance).**